

实验报告 3 —— 网络优化与正则化【严禁抄袭！】

by Kureha Shiro Dec.2025

一、实验目的

- (1) 熟悉掌握常用深层网络的优化和正则化方法。
- (2) 掌握正则化对模型过拟合、优化的作用及特点。
- (3) 使用 MindSpore 验证网络优化与正则化方法的效果。

二、实验原理

2.1 实验数据集

本实验使用的数据集是 Fashion Mnist 数据集, 该数据集的格式和训练测试数据的划分与经典 Mnist 数据集一致。

该数据集包含 70,000 张 28×28 像素的灰度图像, 分为 10 个类别, 每个类别代表一种不同的服装或配饰, 如图 1。

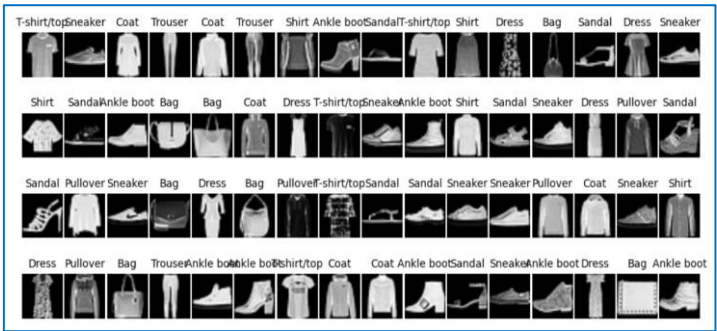


图 1 Fashion Mnist 数据集 (局部)

2.2 实验方法

一般而言, 样本特征由于来源以及度量单位不同, 他们的尺度 (Scale) 往往差异很大。我们对于将全部或部分特征进行缩放后不影响其学习和预测的算法, 称算法具有尺寸不变性。对于特征尺度差异较大时, 梯度下降法的效率也会受其影响, 因此我们需要对其进行数据归一化。

(1) **Batch_Normalization**: 批量归一化, 是一种有效的逐层归一化方法, 可以对神经网络中任意的中间层进行归一化操作。对于深度神经网络:

$$a^{(l)} = f(z^{(l)}) = f(Wa^{(l-1)} + b)$$

批量归一化, 就是使得净输入 $z^{(l)}$ 的分布一致。

机器学习模型的关键是泛化问题, 即在样本真实分布上的期望风险最小化。而训练数据集上的经验风险最小化和期望风险并不一致。由于深度学习的拟合能力特别强, 容易造成模型过拟合, 因此如何提高神经网络的泛化能力反而成为影

响模型能力的关键因素。

(2) **Dropout**: 丢失法, 当训练一个深度神经网络时, 我们可以随机丢失一部分神经元 (同时丢失其对应的连接边) 来避免过拟合。

(3) **L2**: 在损失函数中加入权重参数平方和作为惩罚项, 限制模型参数的大小, 来降低模型复杂度、防止过拟合。

$$\theta^* = \arg \min \frac{1}{N} \sum_{n=1}^N \mathcal{L}(y^{(n)}, f(x^{(n)}; \theta)) + \lambda l_p(\theta)$$

经验风险 正则化项

(4) **Early_Stop**: 提前停止。我们使用一个与训练集独立的样本集合, 成为验证集 (Validation Set), 并用验证集上的错误来代替期望错误, 若测试集上错误率不再下降, 就停止迭代。

三、实验过程及结果

3.1 任务一

运行 **baseline**, 并将不带正则化的结果 (**acc**) 贴至实验报告中。(若任务二也完成, 可与任务二的实验结果合并粘贴)

fc_loss, fc_acc, _ = train(batchnorm=False, dropout=False, l2=False, earlystop=False)
<pre>epoch: 30 step: 54500, loss_train is 0.004332419, acc_test is 0.9086 epoch: 30 step: 54625, loss_train is 0.010763626, acc_test is 0.9059 epoch: 30 step: 54750, loss_train is 5.1803803e-05, acc_test is 0.9051 epoch: 30 step: 54875, loss_train is 1.3259922e-05, acc_test is 0.9097 epoch: 30 step: 55000, loss_train is 0.00018432137, acc_test is 0.9103 epoch: 30 step: 55125, loss_train is 0.003977911, acc_test is 0.9072 epoch: 30 step: 55250, loss_train is 0.0028684326, acc_test is 0.909 epoch: 30 step: 55375, loss_train is 7.057909e-05, acc_test is 0.9095 epoch: 30 step: 55500, loss_train is 0.00036511204, acc_test is 0.9093 epoch: 30 step: 55625, loss_train is 0.03688009, acc_test is 0.9091 epoch: 30 step: 55750, loss_train is 0.007358133, acc_test is 0.91 epoch: 30 step: 55875, loss_train is 0.0006330891, acc_test is 0.9082 epoch: 30 step: 56000, loss_train is 0.0007415757, acc_test is 0.9114 epoch: 30 step: 56125, loss_train is 0.00086225395, acc_test is 0.9122 epoch: 30 step: 56250, loss_train is 5.8859894e-07, acc_test is 0.9084 {'acc': 0.9084}</pre>

图 2 不带正则化的训练结果 (仅截图最后一轮)

fc_loss, fc_acc, _ = train(batchnorm=True, dropout=False, l2=False, earlystop=False)
<pre>epoch: 30 step: 54625, loss_train is 0.0016148521, acc_test is 0.9273 epoch: 30 step: 54750, loss_train is 0.0011352001, acc_test is 0.9279 epoch: 30 step: 54875, loss_train is 0.011000011, acc_test is 0.9258 epoch: 30 step: 55000, loss_train is 0.00031096733, acc_test is 0.9293 epoch: 30 step: 55125, loss_train is 9.8065466e-05, acc_test is 0.9299 epoch: 30 step: 55250, loss_train is 0.0017726487, acc_test is 0.9239 epoch: 30 step: 55375, loss_train is 0.0076345783, acc_test is 0.9264 epoch: 30 step: 55500, loss_train is 3.0260895e-05, acc_test is 0.9276 epoch: 30 step: 55625, loss_train is 8.193797e-05, acc_test is 0.9262 epoch: 30 step: 55750, loss_train is 0.00040388477, acc_test is 0.928 epoch: 30 step: 55875, loss_train is 0.0016425173, acc_test is 0.9298 epoch: 30 step: 56000, loss_train is 0.0024289412, acc_test is 0.9281 epoch: 30 step: 56125, loss_train is 0.00035843157, acc_test is 0.9268 epoch: 30 step: 56250, loss_train is 0.08969922, acc_test is 0.9248 {'acc': 0.9248}</pre>

图 3 带 batchNorm 正则化的训练结果 (仅截图最后一轮)

3.2 任务二

补充代码，实现 Dropout、L2 的正则项的添加，将修改后的网络结构和训练模块代码块、相应模型的训练结果贴至实验报告中。

3.2.1 按照题目要求加入 Dropout

参考 BatchNorm 的实现方式，给下面的网络结构定义中加入 Dropout。

加入正则化前的卷积神经网络，其网络结构为：卷积层 1→卷积层 2→最大池化层→卷积层 3→最大池化层→拉直→全连接层 1→全连接层 2。

Dropout 与 Batch_Normalization 是直接加入网络结构起作用的。我们将其插入至卷积层 2，卷积层 3，全连接层 1 后，网络结构变为：卷积层 1→卷积层 2→最大池化层→(dropout / bn2d(64))→卷积层 3→(dropout / bn2d(128))→最大池化层→拉直→全连接层 1→(dropout / bn1d(128))→全连接层 2。

除了要增加 Dropout 外，原始模型还有个值得改进的地方：BN 和 ReLU 的顺序建议调换。**遵循 Conv->BN->ReLU 的顺序**不仅符合现代深度学习的标准范式（如 ResNet），通常也能带来更稳定的训练效果（如代码段 1）。

代码段 1：在卷积网络结构定义中加入 Dropout，并遵循 Conv->BN->ReLU 的顺序

```
import mindspore.nn as nn
class Identification_Net(nn.Cell):
    def __init__(self, use_dropout=False, use_bn=False):
        super(Identification_Net, self).__init__()
        self.num_class = 10
        self.use_dropout = use_dropout
        self.use_bn = use_bn

        # 1. 定义卷积层
        # Padding 模式设为 valid，与原文档保持一致以匹配维度
        self.conv1 = nn.Conv2d(1, 32, kernel_size=3, stride=1, padding=0,
                                has_bias=False, pad_mode="valid")
        self.conv2 = nn.Conv2d(32, 64, kernel_size=3, stride=1, padding=0,
                                has_bias=False, pad_mode="valid")
        self.conv3 = nn.Conv2d(64, 128, kernel_size=3, stride=1, padding=0,
                                has_bias=False, pad_mode="valid")

        # 2. 定义功能层
        self.maxpool2d = nn.MaxPool2d(kernel_size=2, stride=2)
        self.relu = nn.ReLU()
        self.flatten = nn.Flatten()

        # 3. 定义全连接层
        # 输入维度计算：28 -> 26(conv1) -> 24(conv2) -> 22(conv3) -> 11(pool)
        self.fc1 = nn.Dense(128 * 11 * 11, 128)
```

```

self.fc2 = nn.Dense(128, self.num_class)

# 4. 定义 Batch Normalization 层
self.bn1 = nn.BatchNorm2d(64) # 对应 conv2 输出通道
self.bn2 = nn.BatchNorm2d(128) # 对应 conv3 输出通道
self.bn3 = nn.BatchNorm1d(128) # 对应 fc1 输出神经元

# 5. 定义 Dropout 层：根据题目要求 dropout_ratio=0.5
self.dropout = nn.Dropout(p=0.5)

def construct(self, x):
    # --- 第一层 Block (Conv1) ---
    x = self.conv1(x)
    x = self.relu(x)

    # --- 第二层 Block (Conv2) ---
    x = self.conv2(x)

    # 优化顺序：先 BN 再 ReLU
    if self.use_bn:
        x = self.bn1(x)
    x = self.relu(x)
    # 插入 Dropout
    if self.use_dropout:
        x = self.dropout(x)

    # --- 第三层 Block (Conv3) ---
    x = self.conv3(x)
    # 优化顺序：先 BN 再 ReLU
    if self.use_bn:
        x = self.bn2(x)
    x = self.relu(x)
    # 插入 Dropout（根据文档图示，放在池化前）
    if self.use_dropout:
        x = self.dropout(x)
    x = self.maxpool2d(x)

    # --- Flatten ---
    x = self.flatten(x)

    # --- 第四层 Block (FC1) ---
    x = self.fc1(x)
    # 优化顺序：先 BN 再 ReLU
    if self.use_bn:

```

```

        x = self.bn3(x)
    x = self.relu(x)
    # 插入 Dropout
    if self.use_dropout:
        x = self.dropout(x)

    # --- 输出层 (FC2) ---
    x = self.fc2(x)

    return x

```

3.2.2 按照题目要求增加 L2 正则化

在网络训练时增加 L2 正则项（如代码段 2）。

注：L2 正则化模块需修改 mindspore.nn.Adam，查阅其文档中关于 L2 penalty 的内容；L2 正则化惩罚数值为 $1e-5$ 。

代码段 2：在 train 中增加 L2 正则化（根据 l2 设定 weight_decay）

```

def train(batchnorm=False, dropout=False, l2=False, earllystop=False):
    # 🙋 括号只表默认传参，但我们现在考虑 True 的情况

    cfg.count = 0
    cfg.stop = False
    ds_train, ds_test = create_dataset()
    # 构建网络
    network = Identification_Net(use_bn=batchnorm, use_dropout=dropout)

    # 定义模型的损失函数
    net_loss = nn.SoftmaxCrossEntropyWithLogits(sparse=True, reduction="mean")

    # 【新增】如果 l2 为 True，则设置 weight_decay 为 1e-5，否则为 0.0
    wd = 1e-5 if l2 else 0.0    # 这里我用“A if <条件> else B”作分支，简洁易懂
    net_opt = nn.Adam(network.trainable_params(), learning_rate=cfg.lr, weight_decay=wd)

    # 训练模型
    model = Model(network, loss_fn=net_loss, optimizer=net_opt, metrics={'acc': Accuracy()})

    step_loss_acc_info = StepLossAccInfo(model, ds_test, 125, earllystop)
    print("===== Starting Training =====")
    model.train(30, ds_train, callbacks=[step_loss_acc_info], dataset_sink_mode=False)

    # 验证
    metric = model.eval(ds_test)
    print(metric)
    return mindspore.ops.Stack()(step_loss_acc_info.loss).asnumpy(),
        np.array(step_loss_acc_info.acc), metric

```

完成上述修改后，分别运行增加了 Dropout 和 L2 的两段训练程序，运行结果分别如图 4、图 5。

```
dropout_loss, dropout_acc, _ = train(batchnorm=False, dropout=True, l2=False, earlystop=False)

epoch: 30 step: 54500, loss_train is 0.099829875, acc_test is 0.9253
epoch: 30 step: 54625, loss_train is 0.35081315, acc_test is 0.9274
epoch: 30 step: 54750, loss_train is 0.052500658, acc_test is 0.926
epoch: 30 step: 54875, loss_train is 0.1776575, acc_test is 0.9245
epoch: 30 step: 55000, loss_train is 0.09901941, acc_test is 0.9251
epoch: 30 step: 55125, loss_train is 0.0771335, acc_test is 0.9215
epoch: 30 step: 55250, loss_train is 0.28897232, acc_test is 0.9243
epoch: 30 step: 55375, loss_train is 0.14172895, acc_test is 0.9222
epoch: 30 step: 55500, loss_train is 0.14496224, acc_test is 0.9271
epoch: 30 step: 55625, loss_train is 0.08848697, acc_test is 0.9253
epoch: 30 step: 55750, loss_train is 0.11002139, acc_test is 0.9247
epoch: 30 step: 55875, loss_train is 0.34222177, acc_test is 0.927
epoch: 30 step: 56000, loss_train is 0.20461577, acc_test is 0.9226
epoch: 30 step: 56125, loss_train is 0.08714943, acc_test is 0.9283
epoch: 30 step: 56250, loss_train is 0.19550253, acc_test is 0.9204
{'acc': 0.9204}
```

图 4 增加 Dropout 后模型的训练日志

```
l2_loss, l2_acc, _ = train(batchnorm=False, dropout=False, l2=True, earlystop=False)

epoch: 30 step: 54500, loss_train is 7.4317795e-05, acc_test is 0.9123
epoch: 30 step: 54625, loss_train is 0.0003345711, acc_test is 0.915
epoch: 30 step: 54750, loss_train is 2.5506868e-05, acc_test is 0.9146
epoch: 30 step: 54875, loss_train is 4.3381136e-05, acc_test is 0.9136
epoch: 30 step: 55000, loss_train is 0.0013438078, acc_test is 0.9148
epoch: 30 step: 55125, loss_train is 0.21796732, acc_test is 0.9092
epoch: 30 step: 55250, loss_train is 1.7395287e-05, acc_test is 0.9123
epoch: 30 step: 55375, loss_train is 1.3932386e-05, acc_test is 0.9111
epoch: 30 step: 55500, loss_train is 0.00018377579, acc_test is 0.91
epoch: 30 step: 55625, loss_train is 0.06154969, acc_test is 0.9115
epoch: 30 step: 55750, loss_train is 0.009269782, acc_test is 0.9113
epoch: 30 step: 55875, loss_train is 4.1882853e-05, acc_test is 0.9109
epoch: 30 step: 56000, loss_train is 0.000295774, acc_test is 0.9103
epoch: 30 step: 56125, loss_train is 0.0009627569, acc_test is 0.9152
epoch: 30 step: 56250, loss_train is 0.00010833338, acc_test is 0.9161
{'acc': 0.9161}
```

图 5 增加 L2 正则化后模型的训练日志

综上，对四个模型的 Accuracy 整理如下表 1。

表 1 四个模型的 Accuracy 对比

模型（优化/正则化）	Accuracy
原始（Baseline）	0.9084
带 batchNorm 正则化	0.9248
加入 Dropout	0.9204
加入 L2 正则化	0.9161

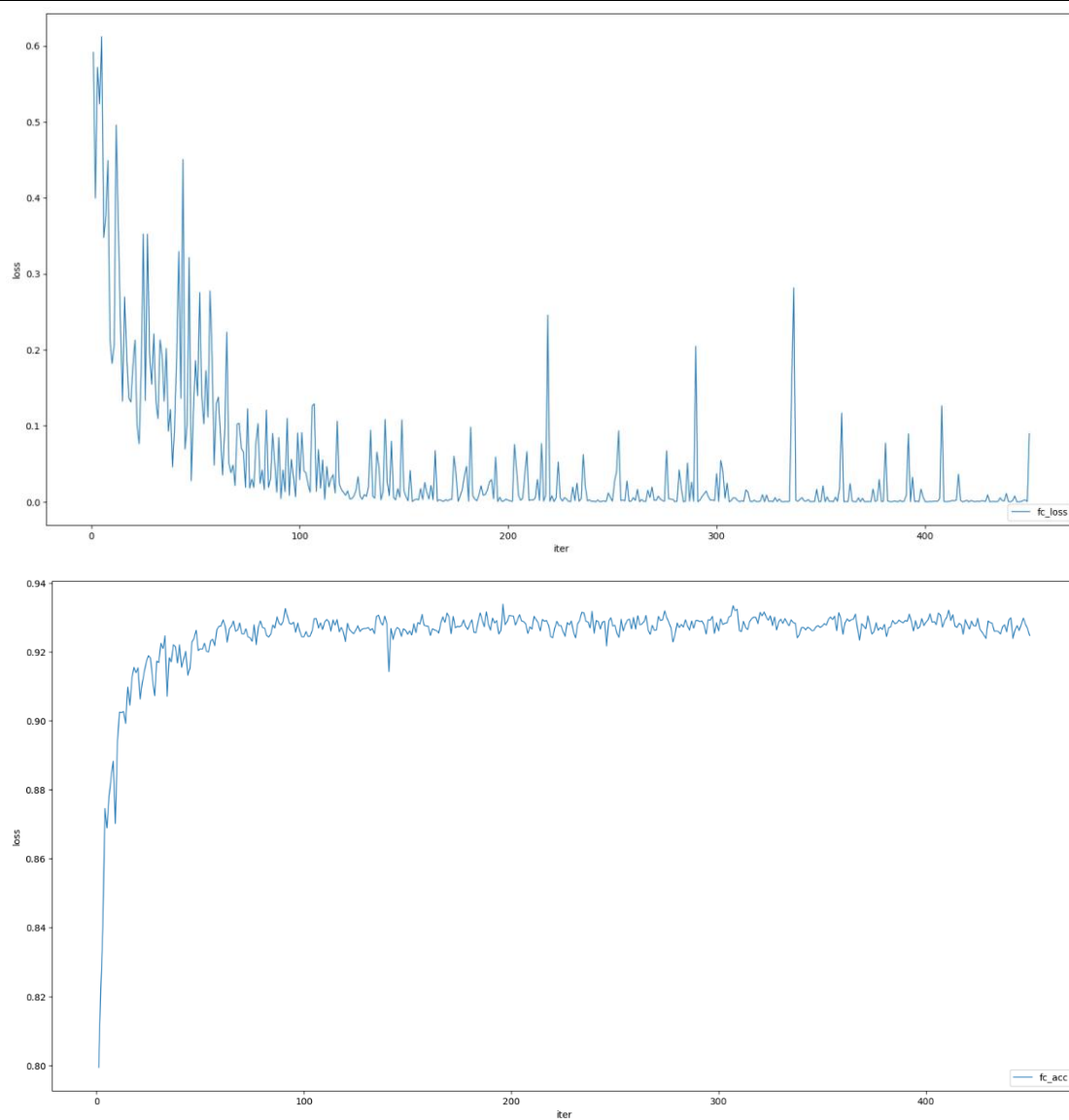
3.3 任务三

请在下方利用 drawCurves 实现对 loss_train 和 acc_test 曲线的绘制，并将结果记录至实验报告中，结合各模型的 acc，试着对结果进行分析。

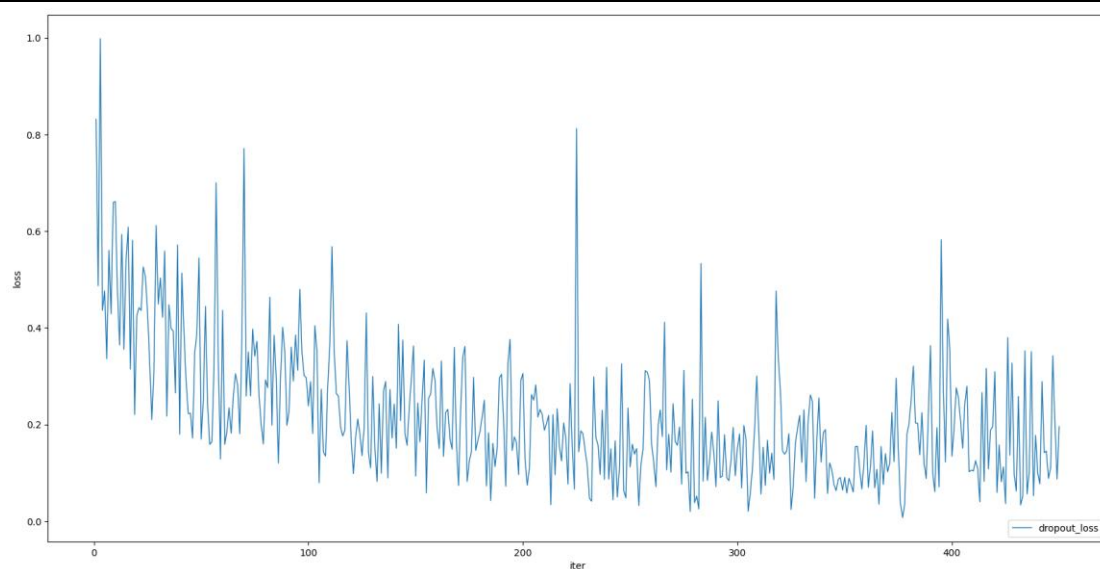
运行程序，得到三种模型（除 Baseline 外）对应的结果如下表 2。

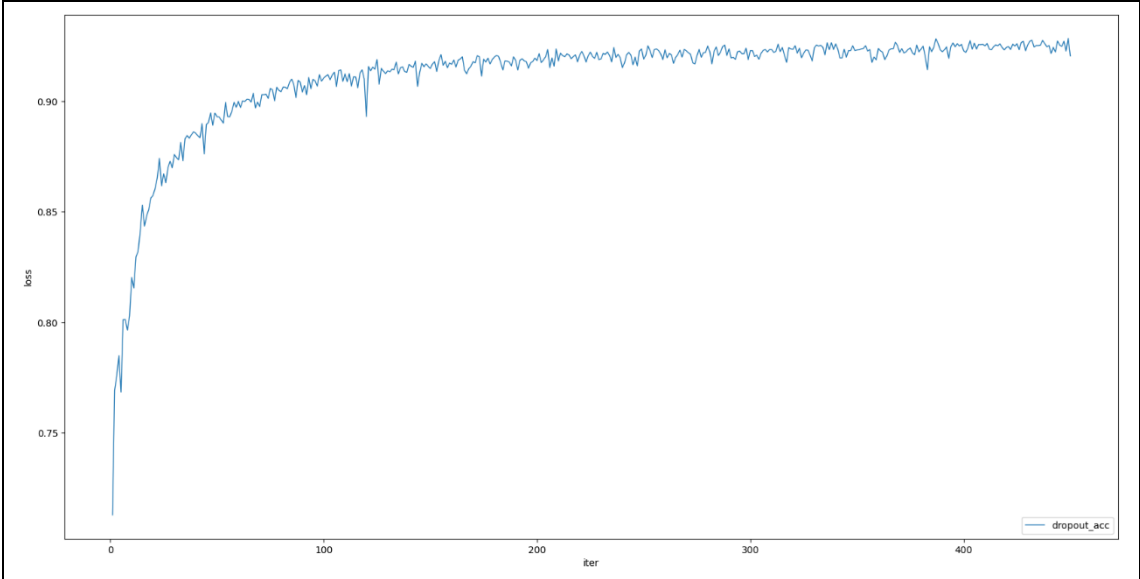
表 2 三种模型 loss_train 和 acc_test 曲线的绘制（从下页起始）

带 batchNorm 正则化

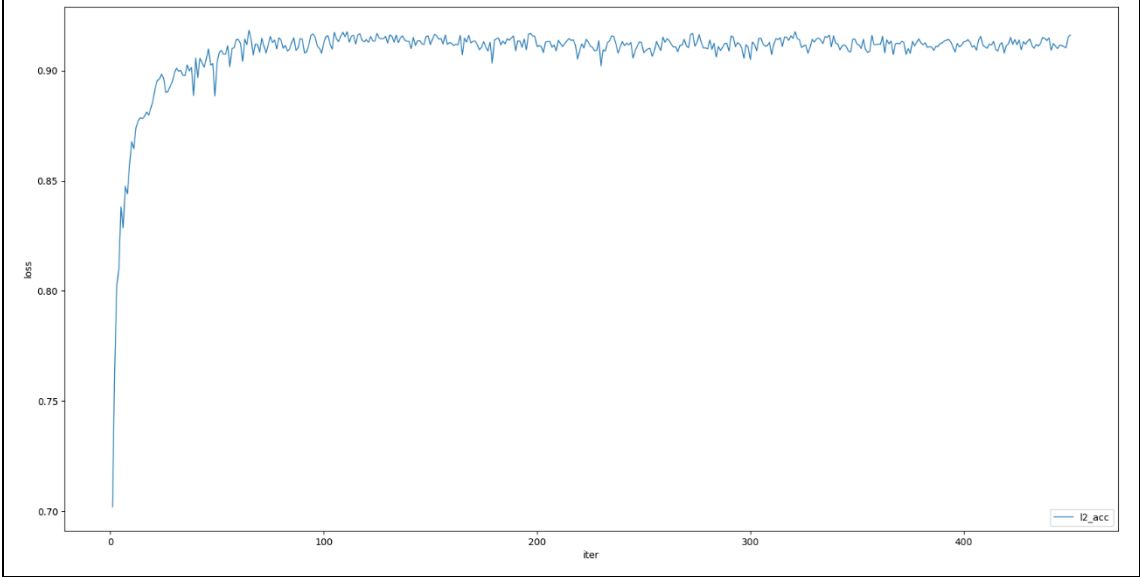
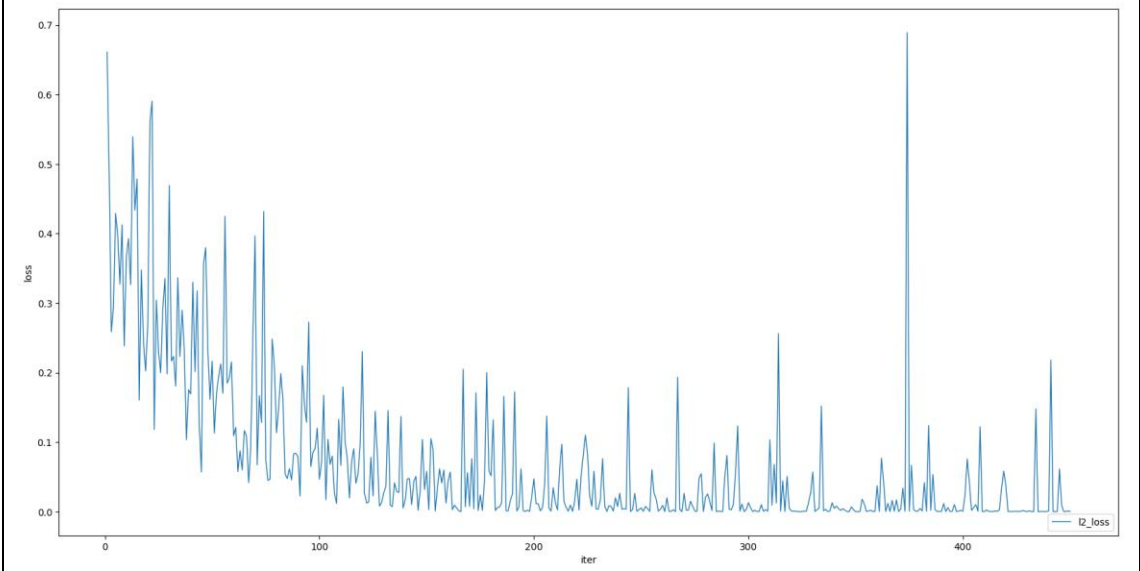


加入 Dropout





加入 L2 正则化



对以上结果图的分析如下：

(1) 带 BatchNorm 正则化的分析

现象描述：从 `loss_train` 曲线来看，收敛速度最快，Loss 值迅速下降并保持在一个较低且稳定的水平，震荡较小。`acc_test` 曲线非常平滑，且在训练早期就达到了较高的准确率。

原因分析：Batch Normalization 通过规范化每一层神经网络的输入，解决了内部协变量偏移（Internal Covariate Shift）问题。这不仅平滑了优化地形，允许模型使用更大的学习率，还起到了正则化的作用。

结论：在本实验中，BatchNorm 的效果最好（Accuracy:0.9248），体现了其加速收敛和提升模型泛化能力的双重优势。

(2) 加入 Dropout 的分析

现象描述：与 BN 相比，Dropout 模型的 `loss_train` 曲线呈现出更明显的震荡（锯齿状）特征，但在后期依然能收敛。`acc_test` 曲线呈上升趋势，最终准确率（0.9204）优于原始模型和 L2 正则化模型。

原因分析：震荡是由于 Dropout 在训练过程中随机“丢弃”部分神经元，导致每次迭代的网络结构略有不同，引入了随机噪声。这种机制强迫网络学习更加鲁棒的特征，防止神经元之间的共适应（Co-adaptation），从而有效抑制过拟合。

结论：Dropout 显著提升了模型的泛化能力，虽然训练过程中的 Loss 波动较大，但最终测试集表现优异，仅次于 BatchNorm。

(3) 加入 L2 正则化的分析

现象描述：`loss_train` 曲线整体呈下降趋势，但伴随一定的波动。`acc_test` 曲线稳步上升，最终准确率（0.9161）高于原始基准模型（0.9084）。

原因分析：L2 正则化通过在 Loss 函数中增加权重参数的平方和（ $\lambda \|\theta\|^2$ ）作为惩罚项，约束了模型参数的大小，使其趋向但不等于 0。限制了模型的复杂度，使其对应的函数曲线更加平滑，从而减少了对训练数据中噪声的过拟合。

结论：L2 正则化有效改善了过拟合，提升了测试集准确率，但在本实验的 Fashion-MNIST 数据集和网络结构下，其提升幅度略逊于 BatchNorm 和 Dropout。

(4) 综合总结

通过对比四种模型（含 Baseline）的实验结果，可以看出所有正则化手段（BN, Dropout, L2）均能提升模型的测试集准确率，验证了正则化在深度学习中防止过拟合的重要性。在本实验的具体场景下，优化效果排序为：

BatchNormalization > Dropout > L2 正则化 > 无正则化(Baseline)

（仅对于本实验而言，并非模型实际“能力”衡量！）

四、实验结论与讨论

4.1 模型改进点

如 [3.2.1 节](#) 所述，遵循 Conv->BN->ReLU 的顺序不仅符合现代深度学习的标准范式（如 ResNet），能带来更稳定的训练效果，这里不再赘述。

4.2 实验结论

本实验在 Fashion-MNIST 数据集上验证了 BatchNormalization、Dropout 和 L2 正则化三种方法对卷积神经网络性能的影响。通过对比不同模型的训练过程（Loss 曲线）和测试集表现（Accuracy），得出以下结论：

（1）正则化手段显著提升了模型泛化能力

实验结果表明，与无正则化的基准模型（Accuracy:0.9084）相比，引入正则化项的三种模型在测试集上的准确率均有提升。这验证了正则化方法在抑制“过拟合”、降低泛化误差方面的核心作用。

（2）不同正则化方法的性能差异

在本实验设定的网络结构和超参数下，**BatchNormalization(BN)**表现最佳。它不仅实现了最高的准确率，而且从 Loss 曲线可以看出，BN 显著加速了模型的收敛速度，并使训练过程更加平滑稳定。

Dropout 表现次之。通过随机丢弃神经元引入噪声，有效破坏了神经元间的共适应性。虽然训练过程中 Loss 震荡较为明显，但最终在测试集上表现出了很强的鲁棒性。

L2 正则化通过惩罚过大的权重参数限制了模型复杂度，也起到了一定的提升效果，但在此次实验中提升幅度略低于 BN 和 Dropout。

（3）网络结构优化的重要性

实验中配合使用的 **Conv→BN→ReLU 的层级顺序**（即在激活函数前进行归一化）被证实是有效的。这种符合现代深度学习范式（如 ResNet）的设计，确保了输入数据在进入非线性激活层之前分布更加稳定，进一步发挥了 BatchNormalization 的效能。

综上所述，合理选择优化器参数（如 WeightDecay）和设计网络结构（加入 BN 层、Dropout 层）是提升深度神经网络性能的关键步骤。在实际应用中，可以结合使用多种正则化策略（如 BN+L2 或 BN+Dropout）以获得最佳效果。